

Appendix A: Fundamentals of Kalman-Filtering Software

Software Details

To facilitate learning, software that is formatted for both IBM- and Macintosh-compatible personal computers containing all of the text's FORTRAN source code listings can be downloaded from the AIAA Web site as described on p. 747. As a special feature for those who work in the MATLAB[®] or True BASIC languages, duplicate files, which are the MATLAB[®] and True BASIC equivalents of the FORTRAN listings, also can be found on the AIAA Web site.

The FORTRAN source code should run as is with either Version 6.0 of the Absoft Power Macintosh Pro FORTRAN compiler or Version 4.3 of the Absoft FORTRAN compiler for IBM-compatible machines. Use of different FORTRAN compilers, in either the Macintosh- or IBM-compatible world, may require some slight modification of the source code. The MATLAB[®] source code should run as is using MATLAB[®] 5.2 on the Macintosh or Version 5.0 on IBM compatible computers. The True BASIC source code should run as is with the Bronze edition on the Macintosh or with the Silver edition on IBM-compatible computers.

The naming conventions for the source code files in each language are slightly different. The FORTRAN naming convention is CxLy.F, where x corresponds to chapter number and y corresponds to listing number. In other words, C4L2.F corresponds to FORTRAN Listing 4.2 of the text (i.e., Chapter 4, Listing 2). The MATLAB[®] naming convention is the same for both Macintosh- and IBM-compatible machines (C4L2.M). In addition, the True BASIC naming convention is also the same for both Macintosh- and IBM-compatible machines (C4L2.TRU).

Each of the source files on the AIAA Web site has a few lines of extra code to make data files so that the user can plot or store the results after a run is made. The name of the generated data file is DATFIL for the Macintosh code and DATFIL.TXT for IBM-compatible code.

The data statements or definition of constants in each of the source code files correspond to those used in the numerical examples presented in the text. The user should first run the program of interest as is in order to verify that the data file generated corresponds to the appropriate figure in the text. Other cases of interest can be run by either changing constants and recompiling or modifying the source code to read input from the keyboard.

In the source code listings that make use of random numbers, use has been made of the FORTRAN uniform random number generators supplied by Absoft. If other FORTRAN compilers are used, the user will have to invoke the appropriate random-number language extension for the particular compiler or may have to write a random-number generator if it is not supplied by the compiler publisher. MATLAB[®] users have special statements for uniform and Gaussian distributions, whereas True BASIC offers a special statement for uniformly distributed random numbers.

MATLAB[®]

Many engineers prefer to work in MATLAB[®] because of its friendly environment and powerful plotting statements. In this Appendix some simple ways of converting the FORTRAN code to MATLAB[®] line by line are presented, and two examples taken from the text are used to illustrate the conversion algorithm. All of the text's source code has also been converted to MATLAB[®] M-files in this way and, as was already mentioned, is included on the AIAA Web site. In addition, the MATLAB[®] code allows the reader to obtain the figures in the book directly without the need for porting the data to other plotting packages. The structure of the translation was originally provided by Dr Michael Dutton of the Weapons Systems Division, Defence Science and Technology Organisation (DSTO) in Salisbury, South Australia, for another book.¹

The method of translation adopted was chosen so that the resultant M-files would closely resemble the original FORTRAN listings. This was done to facilitate a line-by-line conversion and to allow the code to run under earlier MATLAB[®] versions (including the student editions available on both PC and Macintosh platforms) without the need for invoking special library functions and various toolboxes.

MATLAB[®] is an interpretative language, and therefore its code, in general, will run much slower than compiled source code written in FORTRAN. In particular, the use of FOR or WHILE loops are normally avoided in MATLAB[®] because their use dramatically reduces program run-time performance. In this sense no serious attempt was made to optimize the running times of the M-files. However, most of the examples presented in the text took only a few seconds to run using MATLAB[®] on today's powerful desktop computers.

Unfortunately, if the MATLAB[®] code is to mirror closely the FORTRAN listings, the use of FOR and/or WHILE statements becomes necessary. This means that some of the programs will run more than an order of magnitude slower than their FORTRAN counterparts. In general those programs that run in the Monte Carlo mode or those simulations having a small integration interval or long operation times will have significantly longer MATLAB[®] running times. However, even in those cases the worst MATLAB[®] running times were still less than a minute on today's microcomputers.

As an example of how the FORTRAN source code was converted to MATLAB[®], consider the case of Listing 4.4, which originally simulated in FORTRAN a first-order polynomial Kalman filter with gravity compensation. The MATLAB[®] equivalent of Listing 4.4 appears in Listing A.1. We can see that the MATLAB[®] listing is nearly identical to the FORTRAN listing, except that

Listing A.1 MATLAB[®] equivalent of FORTRAN Listing 4.4

```

TS=.1;
PHIS=0.;
A0=400000.;
A1=-6000.;
A2=-16.1;
XH=0.;
XDH=0.;
SIGNOISE=1000.;
ORDER=2;
T=0.;
S=0.;
H=.001;
PHI=[1 TS ;0 1];
P=[99999999 0;0 99999999];
IDNP=eye(ORDER);
Q=zeros(ORDER);
RMAT=SIGNOISE^2;
Q(1,1)=TS*TS*TS*PHIS/3.;
Q(1,2)=.5*TS*TS*PHIS;
Q(2,1)=Q(1,2);
Q(2,2)=PHIS*TS;
HMAT=[1 0];
HT=HMAT;
PHIT=PHI';
count=0;
for T=0:TS:30
    PHIP=PHI*P;
    PHIPPHIT=PHIP*PHIT;
    M=PHIPPHIT+Q;
    HM=HMAT*M;
    HMH=HM*HT;
    HMHTR=HMHT+RMAT;
    HMHTRINV=inv(HMHTR);
    MHT=M*HT;
    K=MHT*HMHTRINV;
    KH=K*HMAT;
    IKH=IDNP-KH;
    P=IKH*M;
    XNOISE=SIGNOISE*randn;
    X=A0+A1*t+A2*T*T;
    XD=A1+2*A2*T;
    XS=X+XNOISE;
    RES=XS-XH-TS*XDH+16.1*TS*TS;
    XH=XH+XDH*TS-16.1*TS*TS+K(1,1)*RES;
    XDH=XDH-32.2*TS+K(2,1)*RES;
    SP11=sqrt(P(1,1));
    SP22=sqrt(P(2,2));
    XHERR=X-XH;
    XDHERR=XD-XDH;

```

(continued)

Listing A.1 (Continued)

```

SP11P=-SP11;
SP22P=-SP22;
count=count+1;
ArrayT(count)=T;
ArrayX(count)=X;
ArrayXH(count)=XH;
ArrayXD(count)=XD;
ArrayXDH(count)=XDH;
ArrayXHERR(count)=XHERR;
ArraySP11(count)=SP11P;
ArrayXDHERR(count)=XDHERR;
ArraySP22(count)=SP22;
ArraySP22P(count)=SP22P;
end
figure
plot(ArrayT,ArrayXHERR,ArrayT,ArraySP11,ArrayT,ArraySP11P),grid
xlabel('Time (Sec)')
ylabel('Error in Estimate of Altitude (Ft)')
axis([0 30 -1500 1500])
figure
plot(ArrayT,ArrayXDHERR,ArrayT,ArraySP22,ArrayT,ArraySP22P),grid
xlabel('Time (Sec)')
ylabel('Error in Estimate of Velocity (Ft/Sec)')
axis([0 30 -200 200])
clc
output=[ArrayT',ArrayX',ArrayXH',ArrayXD',ArrayXDH'];
save datfil output -ascii
output=[ArrayT',ArrayXHERR|1,ArraySP11',ArraySP11P',ArrayXDHERR',ArraySP22',
        ArraySP22P'];
save covfil output -ascii
disp 'simulation finished'

```

semicolons have been added to the end of each line and the output has been saved as individual arrays. At the end of the program, the resultant data are plotted by using MATLAB[®] statements, and the data are sent to ascii files called datfil and covfil. Use of special MATLAB[®] statements to simplify the writing of matrices has also been incorporated in the simulation. In addition, the matrix Riccati equations appear in a more efficient and compact form in the MATLAB[®] listing because subroutines for matrix algebra are no longer required. The FORTRAN Gaussian noise subroutine has been eliminated because MATLAB[®] Gaussian random numbers are automatically generated with the random statement.

The simulation of Listing A.1 was run, and data concerning both the estimates and errors in the estimates were automatically written to the files datfil and covfil. In addition, the plots of errors in the estimates of altitude and velocity were automatically generated with the preceding MATLAB[®] code and appear in Figs. A.1–A.2. Notice that Fig. A.1 is equivalent to FORTRAN-generated results of

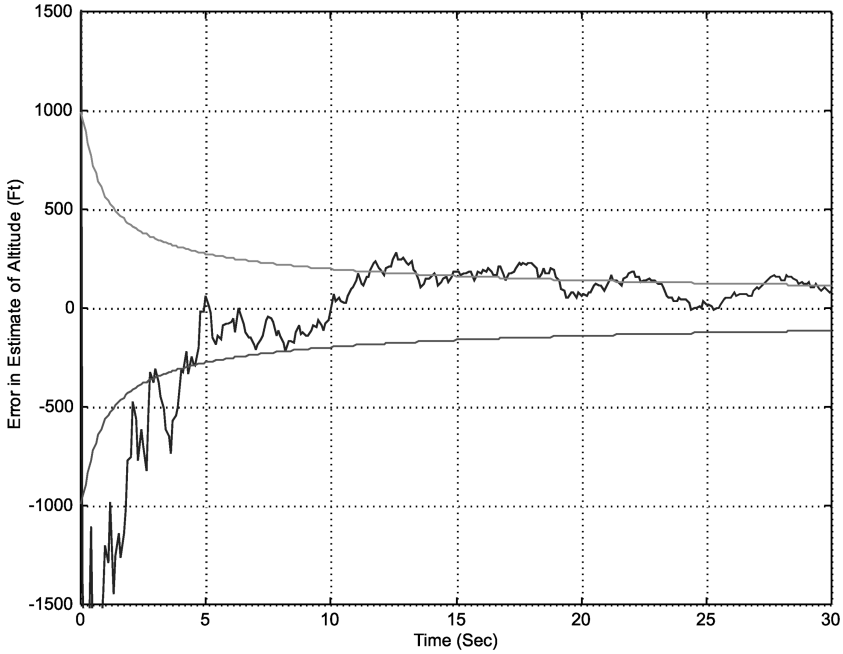


Fig. A.1 MATLAB[®] output is equivalent to Fig. 4.35.

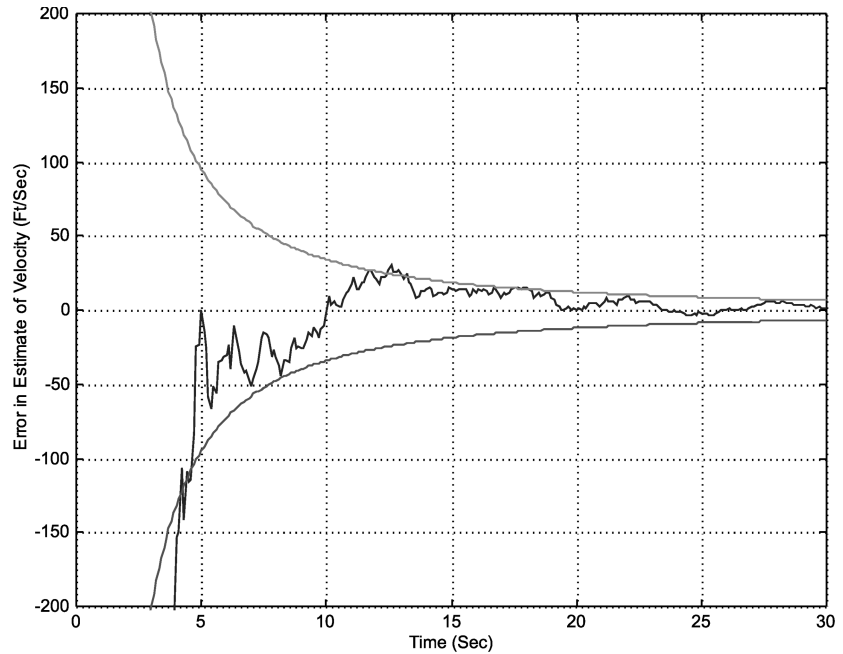


Fig. A.2 MATLAB[®] output is equivalent to Fig. 4.36.

Fig. 4.35, whereas Fig. A.2 is equivalent to the FORTRAN-generated results of Fig. 4.36.

For variety, another FORTRAN example was chosen for conversion because it contained a subroutine. Listing 7.5, which represented an extended Kalman filter with a special subroutine for state propagation, was converted from FORTRAN to MATLAB[®]. The main program that was converted to MATLAB[®] appears in Listing A.2, whereas the equivalent of the FORTRAN subroutine PROJECT appears as a separate M-file, known as PROJECT1.M, in Listing A.3.

The simulation of Listing A.2 was run, and data concerning the estimates and the errors in the estimates were automatically written to the files datfil and covfil. In addition, the few lines of extra code in Listing A.2 automatically generated the plots of Figs. A.3–A.4. Figure A.3 is equivalent to the FORTRAN-generated results of Fig. 7.21 whereas Fig. A.4 is equivalent to the FORTRAN-generated results of Fig. 7.22.

Listing A.2 MATLAB[®] equivalent of FORTRAN Listing 7.5

```
SIGNOISE=25.;
X=200000.;
XD=-6000.;
BETA=500.;
XH=200025.;
XDH=-6150.;
ORDER=2;
TS=.1;
TF=30.;
PHIS=0.;
T=0.;
S=0.;
H=.001;
PHI=zeros(ORDER,ORDER);
P=[SIGNOISE*SIGNOISE 0;0 20000.];
IDNP=eye(ORDER);
Q=zeros(ORDER,ORDER);
HMAT=[1 0];
HT=HMAT';
RMAT=SIGNOISE^2;
count=0;
while T<=TF;
    XOLD=X;
    XDOLD=XD;
    XDD=.0034*32.2*XD*XD*exp(-X/22000.)/(2.*BETA)-32.2;
    X=X+H*XD;
    XD=XD+H*XDD;
    T=T+H;
    XDD=.0034*32.2*XD*XD*exp(-X/22000.)/(2.*BETA)-32.2;
    X=.5*(XOLD+X+H*XD);
    XD=.5*(XDOLD+XD+H*XDD);
    S=S+H;
    if S>=(TS-.00001)
```

(continued)

Listing A.2 (Continued)

```

S=0;
RHOH=.0034*exp(-XH/22000.);
F21=-32.2*RHOH*XDH*XDH/(44000.*BETA);
F22=RHOH*32.2*XDH/BETA;
PHI(1,1)=1.;
PHI(1,2)=TS;
PHI(2,1)=F21*TS;
PHI(2,2)=1.+F22*TS;
Q(1,1)=PHIS*TS*TS*TS/3.;
Q(1,2)=PHIS*(TS*TS/2.+F22*TS*TS*TS/3.);
Q(2,1)=Q(1,2);
Q(2,2)=PHIS*(TS+F22*TS*TS+F22*F22*TS*TS*TS/3.);
PHIT=PHI';
PHIP=PHI*P;
PHIPPHIT=PHIP*PHIT;
M=PHIPPHIT+Q;
HM=HMAT*M;
HMHT=HM*HT;
HMHTR=HMHT+RMAT;
HMHTRINV=inv(HMHTR);
MHT=M*HT;
GAIN=HMHTRINV*MHT;
KH=GAIN*HMAT;
IKH=IDNP-KH;
P=IKH*M;
XNOISE=SIGNOISE*randn;
[XB,XDB,XDDB]=PROJECT(T,TS,XH,XDH,BETA);
RES=X+XNOISE-xb;
XH=XB+GAIN(1,1)*RES;
XDH=XDB+GAIN(2,1)*RES;
ERRX=X-XH;
SP11=sqrt(P(1,1));
ERRXD=XD-XDH;
SP22=sqrt(P(2,2));
SP11P=-SP11;
SP22P=SP22;
count=count+1;
ArrayT(count)=T;
ArrayX(count)=X;
ArrayXH(count)=XH;
ArrayXD(count)=XD;
ArrayXDH(count)=XDH;
ArrayERRX(count)=ERRX;
ArraySP11(count)=SP11;
ArraySP11P(count)=SP11P;
ArrayERRXD(count)=ERRXD;
ArraySP22(count)=SP22;
ArraySP22P(count)=SP22P;

```

end

(continued)

Listing A.2 (Continued)

```

end
figure
plot(ArrayT,ArrayERRX,ArrayT,ArraySP11,ArrayT,ArraySP11P),grid
xlabel('Time (Sec)')
ylabel('Error in Estimate of Altitude (Ft)')
axis([0 30 -20 20])
figure
plot(ArrayT,ArrayERRXD,ArrayT,ArraySP22,ArrayT,ArraySP22P),grid
xlabel('Time (Sec)')
ylabel('Error in Estimate of Velocity (Ft/Sec)')
axis([0 30 -5 5])
clc
output=[ArrayT',ArrayX',ArrayXH',ArrayXD',ArrayXDH'];
save datfil output -ascii
output=[ArrayT',ArrayERRX',ArraySP11',ArraySP11P',ArrayERRXD',ArraySP22',
        ArraySP22P|1];
save covfil output -ascii
disp 'simulation finished'

```

True BASIC

To many older engineers BASIC is simply known as “FORTRAN without grief.” BASIC was originally developed in 1963 at Dartmouth College by John Kemeny and Thomas Kurtz. Although many dialects of BASIC were originally implemented on microcomputers during the 1980s and early 1990s, the major non-object-oriented dialect to survive was the original, which is known commercially as True BASIC. True BASIC, like MATLAB[®], is an interpretive language and is, therefore, much slower than a compiled language, such as FORTRAN.

Listing A.3 MATLAB[®] equivalent of FORTRAN subroutine PROJECT in Listing 7.5

```

function[XH,XDH,XDDH]=project1(TP,TS,XP,XDP,BETA)
T=0.;
X=XP;
XD=XDP;
H=.001;
while T<=(TS-.0001)
    XDD=.0034*32.2*XD*XD*exp(-X/22000.)/(2.*BETA)-32.2;
    XD=XD+H*XDD;
    X=X+H*XD;
    T=T+H;
end
XH=X;
XDH=XD;
XDDH=XDD;

```

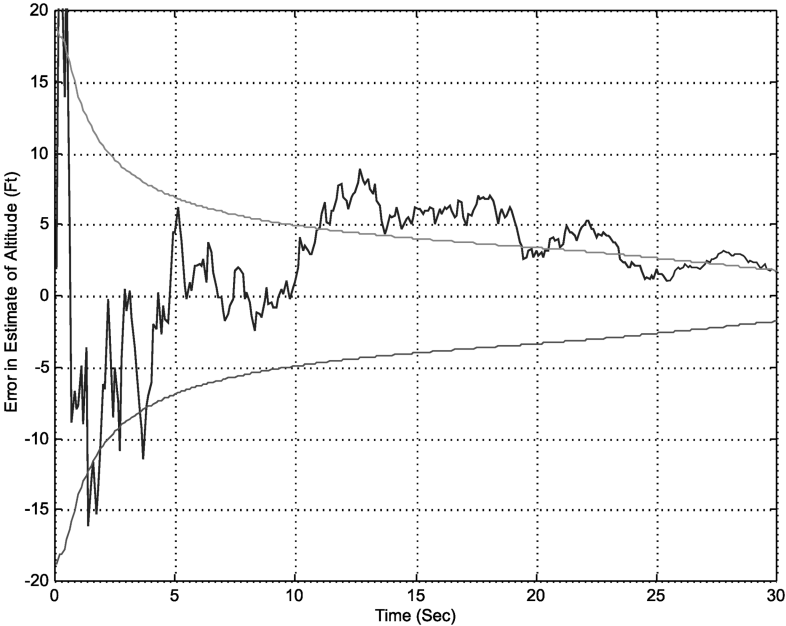


Fig. A.3 MATLAB[®] output is equivalent to Fig. 7.21.

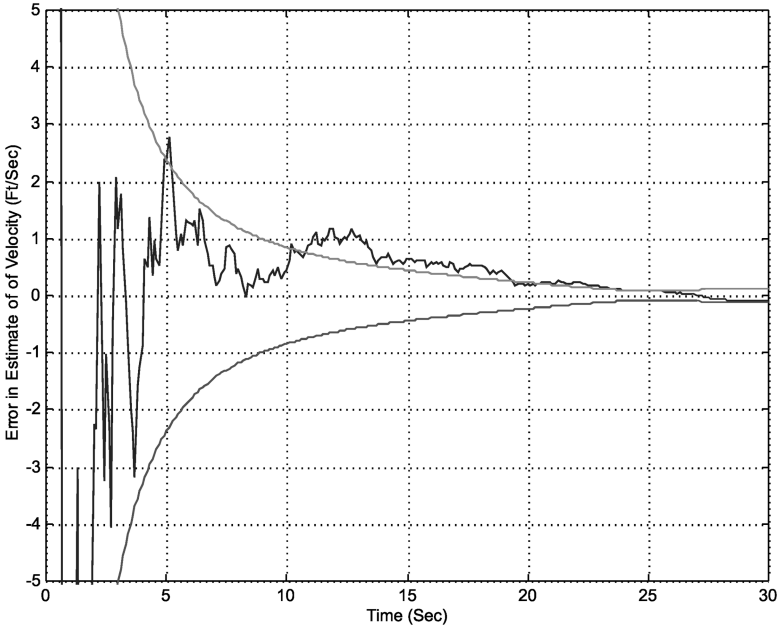


Fig. A.4 MATLAB[®] output is equivalent to Fig. 7.22.

However, as with MATLAB[®], all of the programs in the text take only a few seconds to a minute of running time with True BASIC on today's powerful microcomputers. Its low cost (i.e., approximately \$40 for the regular version and \$20 for the student version) and ease of use also make it a natural choice for implementing the code in this text on personal computers or workstations. As with MATLAB[®], True BASIC also has special functions for matrix manipulation. Therefore, many of the FORTRAN subroutines presented in the text are not required with True BASIC. With a few minor differences the True BASIC code is nearly identical to the FORTRAN code.

As an example of how the FORTRAN source code was converted to True BASIC, let us again consider the case of Listing 4.4, which simulated a first-order polynomial Kalman filter with gravity compensation. The True BASIC equivalent of Listing 4.4 appears in Listing A.4. We can see that the True BASIC listing is virtually identical to FORTRAN Listing 4.4. At the end of the program, the resultant data are sent to ascii files called DATFIL and COVFIL. Use of special True BASIC statements to simplify the writing of matrices also has been incorporated in the simulation. In addition, the matrix Riccati equations appear in a more efficient form the True BASIC listing because subroutines for matrix algebra are no longer required. Unlike MATLAB[®], Gaussian random numbers are not automatically available with True BASIC. However, True BASIC provides a random-number generator yielding uniformly distributed numbers between 0 and 1 with the statement RND. A subroutine, similar to the FORTRAN subroutine GAUSS, makes use of the random-number generator in order to provide zero-mean Gaussian-distributed random numbers.

Listing A.4. True BASIC equivalent of Listing 4.4

```

OPTION NOLET
REM UNSAVE "DATFIL"
REM UNSAVE "COVFIL"
OPEN #1:NAME "DATFIL",ACCESS OUTPUT,CREATE NEW,ORGANIZATION
TEXT
OPEN #2:NAME "COVFIL",ACCESS OUTPUT,CREATE NEW,ORGANIZATION
TEXT
SET #1: MARGIN 1000
SET #2: MARGIN 1000
DIM P(2,2),Q(2,2),M(2,2),PHI(2,2),HMAT(1,2),HT(2,1),PHIT(2,2)
DIM RMAT(1,1),IDNP(2,2),PHIP(2,2),PHIPPHIT(2,2),HM(1,2)
DIM HMHT(1,1),HMHTR(1,1),HMHTRINV(1,1),MHT(2,1),K(2,1)
DIM KH(2,2),IKH(2,2)
TS=.1
PHIS=0.
A0=400000.
A1=-6000.
A2=-16.1
XH=0.
XDH=0.
SIGNOISE=1000.
ORDER=2

```

(continued)

APPENDIX: FUNDAMENTALS OF KALMAN-FILTERING SOFTWARE 821

Listing A.4 (Continued)

```

T=0.
S=0.
H=.001
MAT PHI=ZER(ORDER,ORDER)
MAT P=ZER(ORDER,ORDER)
MAT IDNP=IDN(ORDER,ORDER)
MAT Q=ZER(ORDER,ORDER)
RMAT(1,1)=SIGNOISE^2
P(1,1)=99999999999.
P(2,2)=99999999999.
PHI(1,1)=1.
PHI(1,2)=TS
PHI(2,2)=1.
Q(1,1)=TS*TS*TS*PHIS/3.
Q(1,2)=.5*TS*TS*PHIS
Q(2,1)=Q(1,2)
Q(2,2)=PHIS*TS
HMAT(1,1)=1.
HMAT(1,2)=0.
FOR T=0 TO 30 STEP TS
  MAT PHIT=TRN(PHI)
  MAT HT=TRN(HMAT)
  MAT PHIP=PHI*P
  MAT PHIPPHIT=PHIP*PHIT
  MAT M=PHIPPHIT+Q
  MAT HM=HMAT*M
  MAT HMHT=HM*HT
  MAT HMT=HMHT+RMAT
  HMTINVT(1,1)=1./HMT(1,1)
  MAT MHT=M*HMTINVT
  MAT K=MHT*HMTINVT
  MAT KH=K*HMAT
  MAT IKH=IDNP-KH
  MAT P=IKH*M
  CALL GAUSS(XNOISE,SIGNOISE)
  X=A0+A1*T+A2*T*T
  XD=A1+2*A2*T
  XS=X+XNOISE
  RES=XS-XH-TS*XD+16.1*TS*TS
  XH=XH+XD*TS-16.1*TS*TS+K(1,1)*RES
  XD=XD-32.2*TS+K(2,1)*RES
  SP11=SQR(P(1,1))
  SP22=SQR(P(2,2))
  XHERR=X-XH
  XDHERR=XD-XDH
  PRINT T,X,XH,XD,XDH
  PRINT #1:T,X,XH,XD,XDH
  PRINT #2:T,XHERR,SP11,-SP11,XDHERR,SP22, - SP22
NEXT T

```

(continued)

Listing A.4 (Continued)

```

CLOSE #1
CLOSE #2
END

SUB GAUSS(X,SIG)
LET X=RND+RND+RND+RND+RND-3
LET X=1.414*X*SIG
END SUB

```

True BASIC has a problem creating a new file if an old file with the same name already exists. Therefore, the user must be cautious when running True BASIC several times in a row when data files are automatically created. Either the user must manually delete previously created data files or have statements in the program automatically deleting the files. Listing A.4 has two statements at the beginning of the program to delete previously created files DATFIL and COVFIL. These statements are initially commented out by use of the REM statement. After the program is run once, the word REM can be deleted from the listing two times so that the files of the previous run will automatically be removed.

The True BASIC simulation of Listing A.4 was run, and data concerning the errors in the estimates were automatically written to the file COVFIL. The error in the estimates of altitude and velocity are displayed in Figs. A.5 and A.6, respectively. Figure A.5 is equivalent to the FORTRAN-generated results of Fig. 4.35, whereas Fig. A.6 is equivalent to the FORTRAN-generated results of Fig. 4.36. These results also agree with the MATLAB[®]-generated results of the preceding section (see Figs. A.1 and A.2).

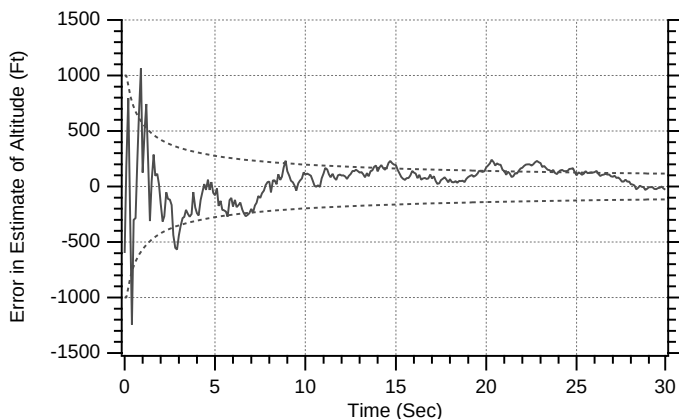


Fig. A.5 True BASIC output is equivalent to Fig. 4.35.

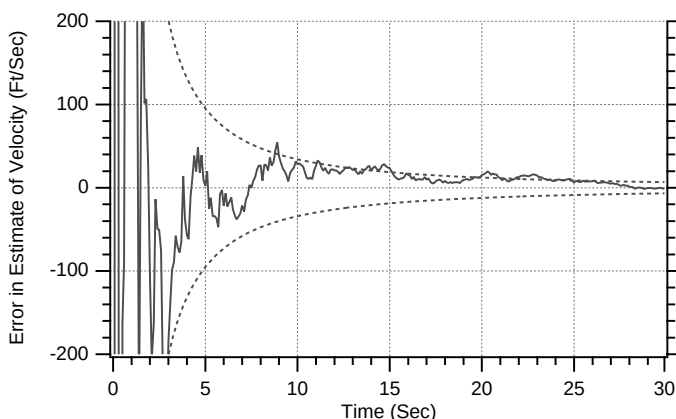


Fig. A.6 True BASIC output is equivalent to Fig. 4.36.

For completeness, Listing 7.5 was also converted to True BASIC and appears in Listing A.5. We can see that with the exception of the matrix operations the True BASIC listing appears to be nearly identical to the FORTRAN listing.

The simulation of Listing A.5 was run, and data concerning the errors in the estimates were automatically written to the file COVFIL. The information in this file was plotted, and the results appear in Figs. A.7 and A.8. Figure A.7 is equivalent to the FORTRAN-generated results of Fig. 7.21, whereas Fig. A.8 is equivalent to the FORTRAN-generated results of Fig. 7.22. These results are also equivalent to the MATLAB[®]-generated results of the preceding section (see Figs. A.3 and A.4).

Listing A.5 True BASIC equivalent of Listing 7.5

```

OPTION NOLET
REM UNSAVE "DATFIL"
REM UNSAVE "COVFIL"
OPEN #1:NAME "DATFIL",ACCESS OUTPUT,CREATE NEW,ORGANIZATION
TEXT
OPEN #2:NAME "COVFIL",ACCESS OUTPUT,CREATE NEW,ORGANIZATION
TEXT
SET #1: MARGIN 1000
SET #2: MARGIN 1000
DIM PHI(2,2),P(2,2),M(2,2),PHIP(2,2),PHIPPHIT(2,2),GAIN(2,1)
DIM Q(2,2),HMAT(1,2),HM(1,2),MHT(2,1)
DIM PHIT(2,2)
DIM HMHT(1,1),HT(2,1),KH(2,2),IDNP(2,2),IKH(2,2)
SIGNOISE=25.
X=200000.
XD=-6000.
BETA=500.

```

(continued)

Listing A.5 (Continued)

```

XH=200025.
XDH=-6150.
ORDER=2
TS=.1
TF=30.
PHIS=0.
T=0.
S=0.
H=.001
MAT PHI=ZER(ORDER,ORDER)
MAT P=ZER(ORDER,ORDER)
MAT IDNP=IDNP(ORDER,ORDER)
MAT Q=ZER(ORDER,ORDER)
P(1,1)=SIGNOISE*SIGNOISE
P(2,2)=20000.
MAT HMAT=ZER(1,ORDER)
MAT HT=ZER(ORDER,1)
HMAT(1,1)=1.
HT(1,1)=1.
DO WHILE T<=TF
    XOLD=X
    XDOLD=XD
    XDD=.0034*32.2*XD*XD*EXP(-X/22000.)/(2.*BETA)-32.2
    X=X+H*XD
    XD=XD+H*XDD
    T=T+H
    XDD=.0034*32.2*XD*XD*EXP(-X/22000.)/(2.*BETA)-32.2
    X=.5*(XOLD+X+H*XD)
    XD=.5*(XDOLD+XD+H*XDD)
    S=S+H
    IF S>=(TS-.00001) THEN
        S=0.
        RHOH=.0034*EXP(-XH/22000.)
        F21=-32.2*RHOH*XDH*XDH/(44000.*BETA)
        F22=RHOH*32.2*XDH/BETA
        PHI(1,1)=1.
        PHI(1,2)=TS
        PHI(2,1)=F21*TS
        PHI(2,2)=1.+F22*TS
        Q(1,1)=PHIS*TS*TS*TS/3.
        Q(1,2)=PHIS*(TS*TS/2.+F22*TS*TS*TS/3.)
        Q(2,1)=Q(1,2)
        Q(2,2)=PHIS*(TS+F22*TS*TS+F22*TS*TS*TS/3.)
        MAT PHIT=TRN(PHI)
        MAT PHIP=PHI*P
        MAT PHIPPHIT=PHIP*PHIT
        MAT M=PHIPPHIT+Q
        MAT HM=HMAT*M
        MAT HMHT=HM*HT
        HMMHTR=HMHT(1,1)+SIGNOISE*SIGNOISE

```

(continued)

APPENDIX: FUNDAMENTALS OF KALMAN-FILTERING SOFTWARE 825

Listing A.5 (Continued)

```

      HMMHTRINV=1./HMMHTR
      MAT MHT=M*HT
      MAT GAIN=HMMHTRINV*MHT
      MAT KH=GAIN*HMMAT
      MAT IKH=IDNP=KH
      MAT P=IKH*M
      CALL GAUSS(XNOISE,SIGNOISE)
      CALL PROJECT(T,TS,XH,XDH,BETA,XB,XDB,XDDDB)
      RES=X+XNOISE-XB
      XH=XB+GAIN(1,1)*RES
      XDH=XDB+GAIN(2,1)*RES
      ERRX=X-XH
      SP11=SQR(P(1,1))
      ERRXD=XD-XDH
      SP22=SQR(P(2,2))
      PRINT T,X,XH,XD,XDH
      PRINT #1:T,X,XH,XD,XDH
      PRINT #2:T,ERRX,SP11,-SP11,ERRXD,SP22,-SP22
    END IF
  LOOP
  CLOSE #1
  CLOSE #2
  END

  SUB GAUSS(X,SIG)
  LET X=RND+RND+RND+RND+RND+RND-3
  LET X=1.414*X*SIG
  END SUB
  SUB PROJECT(TP,TS,XP,XDP,BETA,XH,XDH,XDDH)
  T=0.
  X=XP
  XD=XDP
  H=.001
  DO WHILE T<=(TS-.0001)
    XDD=.0034*32.2*XD*XD*EXP(-X/22000.)/(2.*BETA)-32.2
    XD=XD+H*XDD
    X=X+H*XD
    T=T+H
  LOOP
  XH=X
  XDH=XD
  XDDH=XDD
  END SUB

```

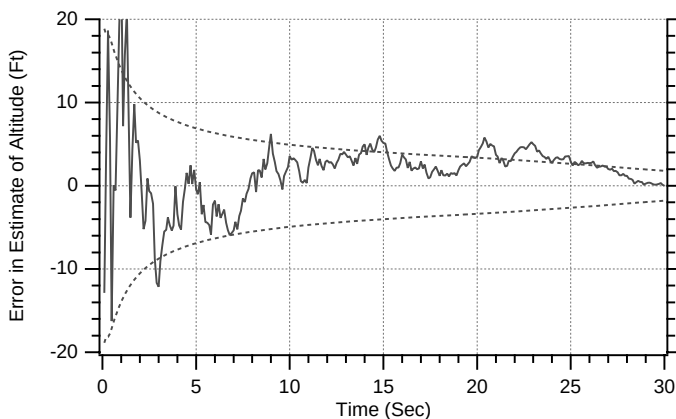


Fig. A.7 True BASIC output is equivalent to Fig. 7.21.

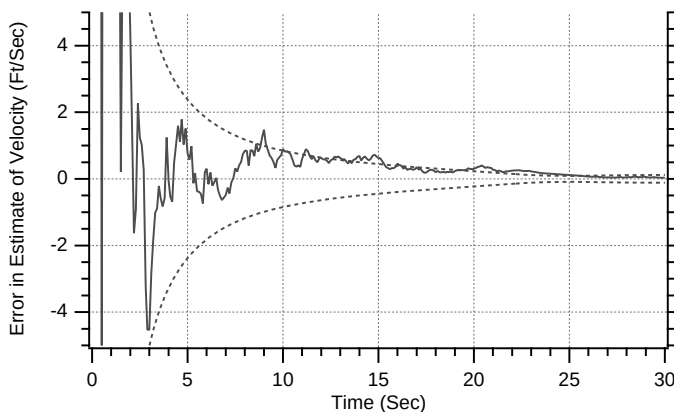


Fig. A.8 True BASIC output is equivalent to Fig. 7.22.

Reference

¹Zarchan, P., *Tactical and Strategic Missile Guidance*, 3rd ed., Progress in Astronautics and Aeronautics, AIAA, Reston, VA, 1998, pp. 585–595.